Case study 1

To develop a PySpark program for integrating healthcare data from multiple file formats such as CSV, JSON, and Parquet by performing structured processing, exploratory analysis, identification of missing values, and data cleaning using appropriate default values to ensure data quality for healthcare analytics applications.

```
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
print("Script Started...")
spark = SparkSession.builder.appName("Healthcare").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
df_csv = spark.read.csv("patient_demographics.csv", header=True,
    inferSchema=True)
df_json = spark.read.json("medical_history.json")
print("CSV Stats:")
df_csv.describe().show()
print("JSON Stats:")
df_json.describe().show()
csv_defaults = {
   "Age": 0,
   "Gender": "Unknown",
   "City": "Unknown"
}
df_csv_clean = df_csv.fillna(csv_defaults)
json_defaults = {
   "History": "None",
   "LastVisit": "2023-01-01"
}

df_json_clean = df_json.fillna(json_defaults)

print("\n--- CSV Data (Before -> After) ---")
```

```
df_csv.show()
df_csv_clean.show()
print("\n--- JSON Data (Before -> After) ---")
df_json.show()
df_json_clean.show()
spark.stop()
```

Case study 2

To implement a comprehensive retail sales data transformation pipeline using PySpark DataFrame operations such as select(), filter(), withColumnRenamed(), withColumn(), drop(), and orderBy() for standardizing column names, filtering transactions, calculating derived metrics, and preparing clean datasets for business reporting and visualization.

```
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
spark = SparkSession.builder.appName("RetailAnalysis").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
df = spark.read.csv("sales_data.csv", header=True, inferSchema=True)
df_renamed = df.select(
    "OrderID",
    "ProdName",
    "Category",
    "Qty",
    "UnitPrice"
).withColumnRenamed(
    "ProdName", "Product_Name"
).withColumnRenamed(
    "UnitPrice", "Price"
)
df_filtered = df_renamed.filter(
    (col("Category") == "Electronics") &
```

```
    (col("Price") > 100)
)
df_final = df_filtered.withColumn(
    "Total_Amount",
    col("Qty") * col("Price")
)
df_sorted = df_final.orderBy(
    col("Total_Amount").desc()
)
print("Final Transformed Retail Sales Data:")
df_sorted.show()
spark.stop()
```

Case study 3

To create a university student feedback analysis system using PySpark GroupBy operations, aggregate functions, filtering, and pivot tables to analyze department-wise and course-wise performance, identify high-rated and under-performing courses, and generate actionable insights for academic improvement.

```
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, count, col
spark = SparkSession.builder.appName("UniversityAnalysis").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
df = spark.read.csv("student_feedback.csv", header=True, inferSchema=True)
print("Department-wise Average Rating:")
df.groupBy("Department").agg(
    avg("Rating").alias("Avg_Rating")
).show()
print("Course-wise Average Rating:")
course_stats = df.groupBy("Course").agg(
    avg("Rating").alias("Avg_Rating"),
    count("Rating").alias("Feedback_Count")
```

```
)
course_stats.show()
print("High-Rated Feedback (Rating > 4):")
df.filter(
    col("Rating") > 4
).show()
print("Pivot Table (Department vs Rating Count):")
df.groupBy("Department").pivot(
    "Rating"
).count().na.fill(0).show()
print("--- Actionable Insights ---")
top_course = course_stats.orderBy(
    col("Avg_Rating").desc()
).first()
poor_course = course_stats.orderBy(
    col("Avg_Rating").asc()
).first()
print(f"Top Course: {top_course['Course']} (Rating: {top_course['Avg_Rating']})")
print(f"Course needing improvement: {poor_course['Course']} (Rating: {poor_course['Avg_Rating']})")
spark.stop()
```

Case study 4

To perform business analytics using PySpark SQL operations by creating temporary views from DataFrames and executing SQL queries such as INNER JOIN, LEFT JOIN, WHERE, GROUP BY, HAVING, ORDER BY, and LIMIT for analyzing customer demographics and transaction relationships.

```
import os
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("BusinessAnalytics").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
```

```
df_cust = spark.read.csv(
    "customers.csv",
    header=True,
    inferSchema=True
)
df_trans = spark.read.csv(
    "transactions.csv",
    header=True,
    inferSchema=True
)
df_cust.createOrReplaceTempView("customers")
df_trans.createOrReplaceTempView("transactions")
print("--- SQL Query Results ---\n")
print("1. Customers with Transactions (INNER JOIN):")
spark.sql("""
    SELECT c.Name, t.Amount, t.Date
    FROM customers c
    INNER JOIN transactions t
    ON c.CustID = t.CustID
""").show()
print("2. All Customers and their spending (including zeros):")
spark.sql("""
    SELECT c.Name, t.Amount
    FROM customers c
    LEFT JOIN transactions t
    ON c.CustID = t.CustID
""").show()
print("3. High-Value Customers (Total Spend > 500):")
spark.sql("""
    SELECT CustID, SUM(Amount) AS Total_Spend
    FROM transactions
    GROUP BY CustID
```

```
    HAVING Total_Spend > 500
    ORDER BY Total_Spend DESC
""").show()
print("4. Top 3 Most Expensive Transactions:")
spark.sql("""
    SELECT TransID, Amount
    FROM transactions
    ORDER BY Amount DESC
    LIMIT 3
""").show()

spark.stop()
```

Case study 5

To build a complete feature engineering pipeline using PySpark MLlib components such as StringIndexer, OneHotEncoder, VectorAssembler, and StandardScaler for preprocessing telecom customer data and preparing normalized feature vectors for machine learning model training.

```
import os
from pyspark.sql import SparkSession
from pyspark.ml.feature import (
    StringIndexer,
    OneHotEncoder,
    VectorAssembler,
    StandardScaler
)
from pyspark.ml import Pipeline
spark = SparkSession.builder.appName("FeatureEngineering").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
df = spark.read.csv(
    "customer_churn.csv",
    header=True,
    inferSchema=True
```

```
)
print("Original Data:")
df.show()

plan_indexer = StringIndexer(
    inputCol="PlanType",
    outputCol="PlanIndex"
)
payment_indexer = StringIndexer(
    inputCol="PaymentMethod",
    outputCol="PaymentIndex"
)
encoder = OneHotEncoder(
    inputCols=["PlanIndex", "PaymentIndex"],
    outputCols=["PlanVec", "PaymentVec"]
)
assembler = VectorAssembler(
    inputCols=[
        "PlanVec",
        "PaymentVec",
        "MonthlyCharges",
        "Tenure"
    ],
    outputCol="features"
)
scaler = StandardScaler(
    inputCol="features",
    outputCol="scaledFeatures"
)
pipeline = Pipeline(
    stages=[
        plan_indexer,
```

```
        payment_indexer,
        encoder,
        assembler,
        scaler
    ]
)
model = pipeline.fit(df)
result = model.transform(df)
print("Final Feature Engineering Results (Scaled Features):")
result.select(
    "CustomerID",
    "features",
    "scaledFeatures"
).show(truncate=False)
spark.stop()
```

Case study 6

To implement a Linear Regression model using PySpark MLlib for predicting house prices based on multiple features, and to evaluate the model performance using RMSE and R^2 metrics for accurate and reliable real estate price prediction.

```
import os
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator

spark = SparkSession.builder.appName("HousePricePrediction").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
df = spark.read.csv(
    "house_prices.csv",
    header=True,
    inferSchema=True
```

```
)
assembler = VectorAssembler(
    inputCols=["SqFt", "Bedrooms", "Age"],
    outputCol="features"
)
data = assembler.transform(df).select(
    "features",
    "Price"
)
train_data, test_data = data.randomSplit(
    [0.8, 0.2],
    seed=42
)
lr = LinearRegression(
    featuresCol="features",
    labelCol="Price"
)
model = lr.fit(train_data)
predictions = model.transform(test_data)
print("Actual vs Predicted Prices:")
predictions.select(
    "Price",
    "prediction"
).show()
evaluator_rmse = RegressionEvaluator(
    labelCol="Price",
    predictionCol="prediction",
    metricName="rmse"
)
evaluator_r2 = RegressionEvaluator(
    labelCol="Price",
    predictionCol="prediction",
```

```
    metricName="r2"
)
rmse = evaluator_rmse.evaluate(predictions)
r2 = evaluator_r2.evaluate(predictions)
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"Coefficient of Determination (R2): {r2:.2f}")

print(f"Coefficients: {model.coefficients}")
print(f"Intercept: {model.intercept}")

spark.stop()
```

Case study 7

To build a Logistic Regression binary classification model using PySpark MLlib for predicting customer churn, and to evaluate model performance using confusion matrix, accuracy, precision, recall, and F1-score for supporting proactive customer retention strategies.

```
import os
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
spark = SparkSession.builder.appName("ChurnPrediction").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

df = spark.read.csv(
    "churn_data.csv",
    header=True,
    inferSchema=True
)
```

```
assembler = VectorAssembler(
    inputCols=[
        "Tenure",
        "MonthlyCharges",
        "TotalCharges"
    ],
    outputCol="features"
)

data = assembler.transform(df).select(
    "features",
    "Churn"
)

train_data, test_data = data.randomSplit(
    [0.7, 0.3],
    seed=42
)
lr = LogisticRegression(
    featuresCol="features",
    labelCol="Churn"
)
model = lr.fit(train_data)
predictions = model.transform(test_data)
print("Actual vs Predicted Churn:")
predictions.select(
    "Churn",
    "prediction",
    "probability"
).show()
print("Confusion Matrix:")
predictions.groupBy(
```

```
    "Churn",
    "prediction"
).count().show()

evaluator = MulticlassClassificationEvaluator(
    labelCol="Churn",
    predictionCol="prediction"
)

accuracy = evaluator.evaluate(
    predictions,
    {evaluator.metricName: "accuracy"}
)
f1 = evaluator.evaluate(
    predictions,
    {evaluator.metricName: "f1"}
)
precision = evaluator.evaluate(
    predictions,
    {evaluator.metricName: "weightedPrecision"}
)
recall = evaluator.evaluate(
    predictions,
    {evaluator.metricName: "weightedRecall"}
)
print(f"Accuracy:  {accuracy:.2f}")
print(f"F1-Score:  {f1:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall:    {recall:.2f}")

print("\nInsight: High probability of churn (label 1) identifies customers at risk.")
```

```
spark.stop()
```

Case study 8

To implement K-Means clustering using PySpark MLlib for customer segmentation based on purchasing behavior, evaluate clustering quality using Silhouette Score, and provide business interpretations and marketing strategies for different customer segments.

```
import os
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

spark = SparkSession.builder.appName("CustomerSegmentation").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

df = spark.read.csv(
    "customer_behavior.csv",
    header=True,
    inferSchema=True
)

assembler = VectorAssembler(
    inputCols=[
        "Spending",
        "Frequency",
        "AccountAge"
    ],
    outputCol="features"
)
data = assembler.transform(df)
```

```
kmeans = KMeans(
    k=3,
    seed=42
)
model = kmeans.fit(data)
predictions = model.transform(data)
print("Customer Cluster Assignments:")

predictions.select(
    "CustomerID",
    "Spending",
    "prediction"
).show()
evaluator = ClusteringEvaluator()
silhouette = evaluator.evaluate(predictions)
print(f"Silhouette Score: {silhouette:.2f}")
print("Cluster Centers (Mean Values for Spending, Frequency, Age):")
centers = model.clusterCenters()
for i, center in enumerate(centers):
    print(f"Cluster {i}: {center}")
print("\n--- Marketing Strategy Recommendations ---")
print("Cluster X (High Spending): Priority for VIP Loyalty Programs.")
print("Cluster Y (Medium Spending): Focus on Up-selling and Cross-selling.")
print("Cluster Z (Low Spending): Re-engagement campaigns and discount
    offers.")
spark.stop()
```